pfi

# Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

# pandas: Combining and Loading, Part 2

WEEK 2, DAY 3, PART 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

# By the end of this part you can

## Report a weighted average

Market-value-weighted OAS and duration.

```
book['oas'] * book['market_value']
```

## Lay out a pivot table

Sector down the side, grade across the top.

```
pd.pivot_table(book, index='sector')
```

## Save results

A CSV file or an Excel workbook.

```
book.to_csv('book.csv', index=False)
```

## Work with dates

Date text as real dates, and a maturity profile.

```
pd.to_datetime(book['maturity'])
```

# A typical bond, or a typical dollar

## Simple average OAS 185.75

Every bond counts the same.

```
book['oas'].mean()
```

```
=AVERAGE(M2:M201)
```

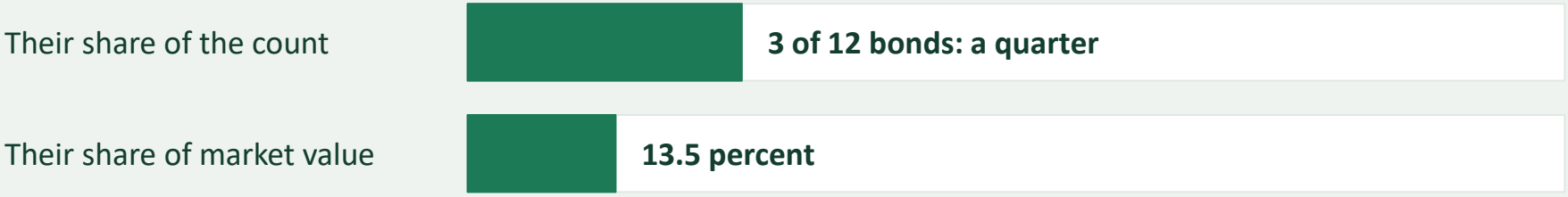
## Weighted average OAS 180.46

Each bond counts in proportion to its market value.

```
(book['oas'] * book['market_value']).sum()
    / book['market_value'].sum()
```

```
=SUMPRODUCT(M2:M201, S2:S201) / SUM(S2:S201)
```

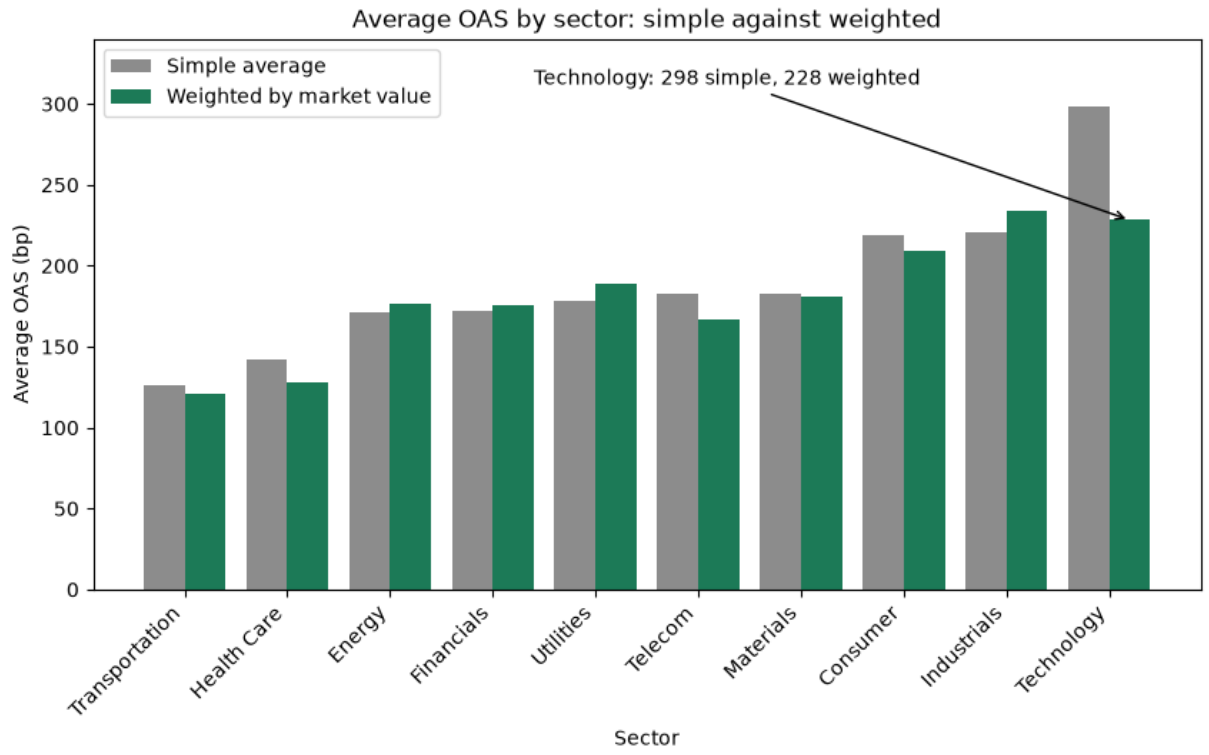
**Technology: 298.2 simple, 228.2 weighted. The three widest of its 12 bonds explain the gap.**




| ticker | rating | oas |
|---|---|---|
| FESR | CCC+ | 1551 |
| FESR | CCC+ | 614 |
| LORE | BB- | 334 |

*Neither figure is wrong. They answer different questions, about an invented portfolio.*

# Simple against weighted, by sector




## Close in most sectors

The grey and green bars are within about 16 bp of each other

## Technology is the exception

298 simple, 228 weighted: a gap of 70 bp

## Why

The wide bonds are small positions, so dollars weight them less

*Read the picture and check it against the numbers in the notebook.*

# A pivot table, a rule, and a file

```
pd.pivot_table(book, values='market_value',
               index='sector', columns='grade',
               aggfunc='sum', margins=True)
```

| sector | High Yield | Investment Grade | All |
|---|---|---|---|
| Materials | 38,747,115 | 32,954,502.5 | 71,701,617.5 |
| Telecom | 27,796,585 | 28,051,685 | 55,848,270 |
| All | 169,901,850 | 320,917,365 | 490,819,215 |

```
book['price_label'] = book['price'].apply(
    price_label)
Discount    113
Premium      87
```

## apply runs a rule on every value

One price in, one label out. In Excel, a nested IF. np.where() gives the same labels on the whole column: prefer it.

Saving: to_csv() writes a table to a file, and read_csv() reads it back

### `book.to_csv(file)`

25 columns saved, 26 read back. The row numbers return as a stray column.

### `index=False`

25 saved, 25 read back. Row numbers carry no information, so leave them out.

### `sector_summary.to_csv(file)`

A grouped table keeps its index. It holds the sector names.

# Identifiers are text: dtype={'cusip': str}

The same file, read two ways

```
as_numbers = pd.read_csv(io.StringIO(typed_file))
0   9900123   1000000
as_text = pd.read_csv(io.StringIO(typed_file),
                      dtype={'cusip': str})
0  009900123   1000000
```

The file holds 009900123. Read as a number, the leading zero is gone, and a merge on that column would match nothing, with no error.

**In Excel** A double-click on the CSV shows 9900123 as well. Data, From Text/CSV, with that column set to Text, keeps the zero.

**The habit: read identifiers as text. A CUSIP, an ISIN, an account number: each is a label, not a quantity.**

# Dates arrive as text

**As loaded: text. It cannot be subtracted.**

```
0    2032-06-18
1    2029-04-17
2    2030-06-03
Name: maturity, dtype: str
```

`pd.to_datetime()` →

**Converted: real dates**

```
0   2032-06-18
1   2029-04-17
2   2030-06-03
Name: maturity, dtype: datetime64[us]
```

The parts, with .dt

| | |
|---|---|
| year | **2032** |
| month | **6** |
| day_name() | **Friday** |

```
days_left = (book['maturity'] - as_of).dt.days
book['years_to_maturity'] = days_left / 365.25
```

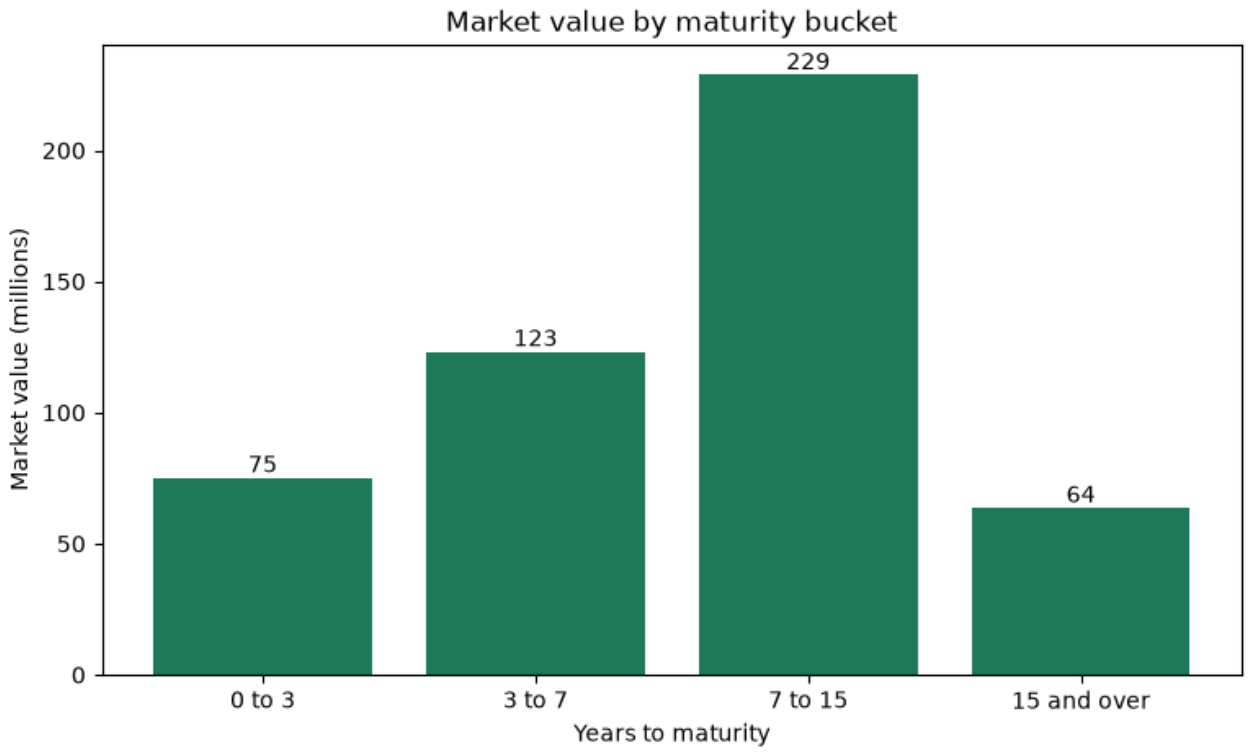
| Shortest | Longest | Average |
|---|---|---|
| 1.0 | 29.9 | 9.85 |

*Years to maturity as of 2026-09-30, by calendar days. An approximation, not the 30/360 count used to price these bonds.*

pd.cut() sorts the years into buckets. Each box is as wide as its share of the 200 bonds.

| 0 to 3 | 3 to 7 | 7 to 15 | 15 and over |
|---|---|---|---|
| 28 bonds | 51 bonds | 88 bonds | 33 bonds |
| 75,125,702.5 | 123,242,385 | 228,787,395 | 63,663,732.5 |

# The maturity profile




### 7 to 15 years is the tallest

About 229 million of the 491 million

### 3 to 7 years follows

About 123 million

### Built from three steps

to_datetime, years to maturity, then pd.cut and groupby

*Read the picture and check it against the numbers in the notebook.*

# Excel workbooks: write, read back, two sheets

```
book.to_excel('desk_report.xlsx',
              sheet_name='book', index=False)
from_excel = pd.read_excel('desk_report.xlsx',
    sheet_name='book', dtype={'cusip': str})
Read back: (200, 30)
Market value read back: 490,819,215
```

What survived the trip: the type of three columns

| column | saved | read back |
|---|---|---|
| cusip | str | str |
| maturity | datetime64[us] | datetime64[us] |
| maturity_bucket | category | str |

*A workbook stores a date as a date, where a CSV stores it as text. The bucket labels survive and their order does not. No formula makes the trip: pandas writes what Paste Special, Values would leave.*

```
with pd.ExcelWriter('desk_report.xlsx') as writer:
    book.to_excel(writer, sheet_name='book', index=False)
    sector_summary.to_excel(writer, sheet_name='by_sector')
book: (200, 30)
by_sector: (10, 5)
```

| pandas | Excel |
|---|---|
| to_excel() | File, Save As, Excel Workbook |
| read_excel() | File, Open, then the sheet's tab |
| pd.ExcelWriter | Two sheets, each with its own tab |

# The AI check

Every notebook ends with code written in the style of an AI assistant. It has one bug. Find it.

1

## Read

Read the code before you run it.

2

## Predict

Decide what it should produce.

3

## Run

Run it and look at the output.

4

## Check

Compare with a number you worked out yourself.

*Code that runs is not the same as code that is right.*

# What you can do now

- Report a market-value-weighted OAS or duration
- Join on the index, and style a comparison
- Lay a summary out as a pivot table, and label bonds with a rule
- Save to a CSV file or an Excel workbook, and read identifiers as text
- Turn date text into dates, and build a maturity profile

**Tomorrow** Case study: three tables from three systems, and a first look at each